Hockey UQ Project — Architecture & Build Plan

A probabilistic hockey prediction pipeline with full uncertainty quantification, structured as a port and extension of the AEM 8290 land-surface column UQ project. The goal is calibrated predictive distributions over game outcomes, player point totals, and (eventually) season-long quantities — not point estimates.

This document is the project's reference architecture and roadmap. Come back to it when you forget what `analysis/` is supposed to contain or why `game_model.py` lives at the top level of `models/`.

Project Goals

The end-state pipeline produces, for any matchup:

- $P(\text{home win})$ with credible intervals
- Predictive distributions over final score, total goals, goal differential
- Predictive distributions over individual player point totals
- Sensitivity decomposition: which inputs drive outcome variance

All of these come from the same underlying machinery — a hierarchical Bayesian model fit to league-wide data, propagated through a game model and Monte Carlo simulator, validated with proper scoring rules.

First shipping target: single-game win probability. Smallest pipeline that exercises every layer.

Status

Last updated: 2026-10-18

Phase	Status	Result
1. Data foundation	**Done**	5 seasons (2021-22 to 2025-26), 6,560 games, full play-by-play, rebound/rush context validated
2. xG model	**Done**	LightGBM, test AUC 0.769, team-season xG correlates 0.97 with MoneyPuck
3. Team strength & goalie aggregates	**Done**	Rolling 25-game xG team strength picks winners 59% of the time; Bayesian goalie rating (k = 800, season-adjusted)
4. Track A toy MCMC	**Next**	
5–8	Not started	
v2. Model improvements	Planned (after Phase 7)	Hierarchical xG layer, lineup (player-level) model — see *v2 — Planned Improvements*

Check current state: `py .\scripts\check_state.py` lists which files and tables exist and whether `shot_context` matches the current derivation code.

The Six-Step Arc

The full pipeline, end-to-end:

1. **Build component models** — xG (gradient boosting), team strength aggregates, goalie ratings. Each produces estimates with uncertainty.
2. **Combine via Poisson/NB game model** — maps team-level inputs to $\lambda_{\text{home}}, \lambda_{\text{away}}$.
3. **Monte Carlo simulation** — propagates uncertainty from every layer into a predictive distribution over outcomes.
4. **Aggregate sims** into the distributions you actually care about: win probabilities, score distributions, player point totals.
5. **Validate** with proper scoring rules (log-loss, Brier, CRPS) and calibration plots on held-out games.
6. **Fit PCE surrogate** over the simulator's input space — enables fast sensitivity analysis and analytical Sobol decomposition.

Mapping to the AEM 8290 Soil-Atmosphere Project

The hockey UQ pipeline is structurally the same as your AEM 8290 land-surface column project, with the forward model swapped and a hierarchical layer added. Most of the conceptual machinery already exists in working code.

Hockey project component	Soil project equivalent
Component models (xG, team strength, goalie)	Forward model: soil heat + Richards' + SEB
Game model ($\lambda_{\text{home}}, \lambda_{\text{away}}$)	Forward model mapping $\theta \rightarrow$ QoIs
Monte Carlo over game outcomes	Forward propagation, screening, Sobol Monte Carlo
Aggregation to predictive distributions	Posterior moments, predictive distributions
Held-out validation, calibration plots	LOO-CV Q^2, held-out RMSE, trace plots, ACF
PCE surrogate for Sobol indices	PCE surrogate for MCMC efficiency

What transfers directly:

- The Adaptive Metropolis sampler (random-walk MH, burn-in / adapt / sample, $(2.38)^2/d$ scaling, $\xi \in [-1,1]^d$ standardized space)
- Additive Gaussian likelihood structure (mechanics — independent QoI noise summed in log-space)
- Objective prior strategy (uniform-bounded for location, log-uniform for scale spanning decades)
- PCE-as-surrogate trick for expensive forward models
- Validation discipline (Q^2, trace plots, acceptance rates, ACF, posterior recovery vs truth)

What changes:

- **Dimensionality.** Soil: 5 calibration parameters. Hockey hierarchical model: ~70–100+ parameters. Random-walk MH does not scale; NUTS (HMC variant via PyMC/NumPyro) required.
- **Hierarchy.** Soil model is flat. Hockey model has population-level hyperpriors with partial pooling across teams and positions.
- **Likelihood form.** Gaussian becomes Poisson (goal counts) or Bernoulli (win/loss).
- **Forward model speed.** Soil: ~1 s/eval, surrogate essential. Hockey game model: milliseconds/eval, surrogate optional. PCE earns its keep in hockey for the **Sobol analysis**, not for inference speed.
- **Data volume.** Soil: synthetic 24–72h time series of 2 QoIs. Hockey: ~1200 games/season × multiple observables per game.

Architecture

Folder Structure

```
hockey-stats/
│
├── pyproject.toml
├── README.md
├── .gitignore
├── .env.example
│
├── config/
│   ├── settings.py             # paths, current season, DB location
│   └── teams.py                # team metadata, colors, abbreviations
│
├── data/                       # SQLite database lives here
│   ├── hockey_stats.db
│   └── external/               # files downloaded by hand (e.g. MoneyPuck teams.csv)
│
├── src/
│   │
│   ├── data/                   # data acquisition & storage
│   │   ├── __init__.py
│   │   ├── nhl_api.py          # NHL API client
│   │   ├── db.py               # SQLite operations
│   │   ├── scraper.py          # Natural Stat Trick / Evolving-Hockey scrapers
│   │   └── cache.py            # caching layer
│   │
│   ├── processing/             # raw data → analysis-ready
│   │   ├── __init__.py
│   │   ├── shot_context.py     # DONE: rebound / rush flags → shot_context table
│   │   ├── xg_processing.py    # DONE: xG feature table, one row per shot
│   │   ├── shots.py            # shot data processing
│   │   ├── corsi.py            # Corsi/Fenwick calculations
│   │   ├── game_score.py       # game score breakdown
│   │   ├── team_strength.py    # DONE: team_game_xg, rolling_strength, strength_as_of
│   │   ├── stints.py           # v2: shifts → stints (constant on-ice sets)
│   │   ├── rapm.py             # RAPM / WAR (future)
│   │   └── goalie.py           # DONE: goalie_game_xg (per-goalie, per-game xGA / GA)
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   ├── schemas.py          # Pydantic: Game, Player, Shot, ScheduleGame
│   │   ├── game_model.py       # TOP-LEVEL: maps team inputs → λ_home, λ_away
│   │   │
│   │   ├── deterministic/
│   │   │   ├── __init__.py
```

```
│   │   │   ├── xg.py           # DONE: ExpectedGoalsModel (LightGBM)
│   │   │   └── rapm.py         # ridge regression RAPM (future)
│   │   │
│   │   └── stochastic/
│   │       ├── __init__.py
│   │       ├── am_sampler.py   # Adaptive Metropolis (ported from soil)
│   │       ├── goalie.py       # DONE: goalie_rating_as_of (Gamma-Poisson, season-adjusted)
│   │       ├── toy_mcmc.py     # Track A: hockey-flavored AM MCMC
│   │       ├── hierarchical.py # Track B: PyMC hierarchical model
│   │       ├── xg_hierarchical.py # v2: Bayesian layer on top of LightGBM xG
│   │       ├── lineup_rapm.py  # v2: Bayesian RAPM player effects
│   │       ├── montecarlo.py   # Monte Carlo over posterior samples
│   │       └── pce.py          # PCE surrogate for Sobol analysis
│   │
│   ├── analysis/               # NEW: reduce sim output to distributions
│   │   ├── __init__.py
│   │   ├── aggregation.py      # P(win), credible intervals, marginals
│   │   └── sensitivity.py      # Sobol indices from PCE coefficients
│   │
│   ├── validation/             # NEW: scoring, calibration, backtesting
│   │   ├── __init__.py
│   │   ├── scoring.py          # log-loss, Brier, CRPS
│   │   ├── calibration.py      # reliability diagrams
│   │   └── backtest.py         # walk-forward validation harness
│   │
│   ├── pipelines/              # NEW: high-level orchestration
│   │   ├── __init__.py
│   │   ├── train_xg.py         # DONE: build features, train, evaluate, save
│   │   ├── score_xg.py         # DONE: score every shot into shot_xg (leave-one-season-out)
│   │   ├── fit_hierarchical.py # fit PyMC model, save ArviZ NetCDF
│   │   ├── fit_toy.py          # fit Track A toy MCMC for comparison
│   │   ├── fit_xg_layer.py     # v2: fit hierarchical xG layer, save NetCDF
│   │   ├── fit_lineup.py       # v2: build stints, fit RAPM, save NetCDF
│   │   ├── predict_game.py     # end-to-end single-game prediction
│   │   ├── backtest_season.py  # run validation harness on held-out games
│   │   └── build_pce.py        # fit PCE surrogate for Sobol
│   │
│   ├── viz/                    # visualization (existing)
│   │   ├── __init__.py
│   │   ├── theme.py
│   │   ├── rink_plot.py        # DONE: draw_rink, plot_game_events, plot_game
│   │   ├── shot_heatmap.py
│   │   ├── game_summary.py     # (future)
│   │   ├── player_card.py      # (future)
│   │   └── components/
│   │
│   └── export/
│       ├── __init__.py
│       └── save.py
│
├── notebooks/                  # exploration and checks
│   ├── data_sanity.ipynb       # raw data checks (sections 1-13)
│   ├── xg_features_check.ipynb # xG feature table checks
│   ├── xg_model_dev.ipynb      # baseline, LightGBM, evaluation, MoneyPuck comparison
│   └── phase3_check.ipynb      # team strength + goalie rating checks and settings
│
├── output/                     # generated images, posterior samples
│   ├── game_summaries/
│   ├── player_cards/
│   ├── posteriors/             # NetCDF posterior samples
│   └── models/                 # pickled trained models (xG); keep out of git
│
├── tests/
│   ├── test_corsi.py
│   ├── test_game_score.py
│   └── fixtures/
│
└── scripts/                    # CLI entry points
    ├── load_all.py                 # teams, rosters, schedule, games, shots, events, shifts
    ├── backfill_game_events.py     # one-time: full play-by-play for loaded games
    ├── rebuild_shot_context.py     # re-derive shot_context after a code change
```

```
    ├── check_state.py                # project state + shot_context sync check
    ├── fix_goal_shooters.py          # one-time repair (goal scorer IDs)
    ├── check_block_attribution.py    # diagnostic: who owns a blocked-shot event
    ├── check_block_zone_frame.py     # diagnostic: which frame blocked-shot zones use
    ├── nst_spotcheck.py              # compare rush/rebound counts with Natural Stat Trick
    └── rush_event_audit.py           # list rush-flagged shots for one game
```

Why Certain Things Live Where They Do

`models/game_model.py` at the top level of `models/` — neither purely deterministic nor purely stochastic. It's the bridge that takes deterministic components and lifts them into a stochastic combiner. Forcing it into one subfolder would be a category error. It belongs alongside `schemas.py` as a top-level model concern.

`analysis/` separate from `models/` — `models/` is about computation that produces parameters or predictions. `analysis/` is about *reducing* simulation output to the distributions you care about. Different responsibility: aggregation, not modeling.

`validation/` separate from everything — scoring rules and calibration plots are infrastructure that applies to any model. Keeping them separate means they can be reused if you swap component models.

`pipelines/` for orchestration — high-level entry points that chain components together. Each pipeline file should read like a recipe: load this, fit that, save the output. No business logic, just glue.

Two tracks under `stochastic/` — `toy_mcmc.py` (Track A, your soil-pipeline port) and `hierarchical.py` (Track B, PyMC) coexist. Track A is the reference implementation; Track B is the production model. They share input contracts so the rest of the pipeline doesn't care which produced the samples.

Data Contract

Tables

Table	Rows	One row per	Notes
`shot_events`	773,259	shot attempt (incl. blocked, shootout)	Raw; filter at query time (below)
`game_events`	2,060,996	play-by-play event of any type	Raw; ordered by `sort_order`, never `event_id`
`shot_context`	566,545	unblocked, non-shootout shot	Derived from `game_events` by `shot_context.py`
`shot_xg`	562,374	scored shot (unblocked, periods 1–4, goalie in net)	Derived by `score_xg.py`; each season scored by a model that did not train on it
`shifts`	4,578,542	player shift	
`games`, `schedule`	6,560	game	1,312 per season × 5

`shot_context` joins to `shot_events` on `(game_id, event_id)`. After any change to `src/processing/shot_context.py`, run `scripts/rebuild_shot_context.py`; `check_state.py` flags a stale table.

Filtering Rules

`shot_events` is stored **raw and complete** — no filtering is applied at the database level. Shootout attempts (period 5) and blocked shots are all present. This is deliberate: the raw table stays a faithful record, and every consumer applies the filters it needs at query time.

Two rules, applied at query time:

1. `period <= 4` — excludes shootout (period 5) events; keeps OT (period 4).
2. `event_type IN ('shot-on-goal', 'goal', 'missed-shot')` — excludes blocked shots (degenerate coordinates) and shootout attempts.

Together these yield **566,545 clean unblocked non-shootout shots**. The xG feature table then drops empty-net shots (opposing goalie digit = 0 in `situation_code`), leaving **562,374 training shots** (Phase 2).

Why this matters per consumer:

- **xG model (Phase 2):** apply both rules, then drop empty-net shots. Blocked shots lack a true shot location, shootout attempts are mislabeled as goals, and empty-net shots score regardless of geometry; any of them would poison training.
- **Game model (Phase 6):** use `period <= 4` goals as the Poisson target so shootout-winner credits don't inflate goal counts. The shootout is modeled separately (flat 50/50 conditional on a tied regulation — see deferred OT/SO decision).

Background: the data-sanity notebook found that period-5 shootout attempts are stored as `event_type` `'goal'` / `'shot-on-goal'` / `'missed-shot'` (1,065 shootout "goals", ~2.6% of all goal events). All score-reconciliation residuals after filtering are off-by-exactly-1 (the shootout-winner credit added to the official score with no corresponding real goal event), zero off-by-more-than-1 — fully characterized and expected, not a bug.

Known Data Quirks (verified against this database)

- **Goal scorers:** the API stores the scorer under `scoringPlayerId`, not `shootingPlayerId`. `parse_shot_events` reads both. 41,536 goals were repaired once with `scripts/fix_goal_shooters.py`.
- **Blocked-shot owner:** `eventOwnerTeamId` is the **shooting** team (90/10, `check_block_attribution.py`).
- **Blocked-shot zone:** `zoneCode` is written from the **defending** team's side even though the owner is the shooter (99.9% of team-periods, `check_block_zone_frame.py`). `shot_context.py` flips it before the rush rule.
- **Zone codes** are relative to the event owner (O / N / D). Compare two events only after translating both into the shooter's frame.
- **`situation_code`** is 4 digits: away goalie, away skaters, home skaters, home goalie (`1551` = 5v5). Decode from the shooter's point of view.

- **Shot-type labels shifted in 2025-26** (many wrist → snap, deflected → tip-in). The xG features group them as `wrist-snap` and `redirect`.
- **Missing shot type** occurs almost only on goals; filling it with "unknown" leaks the outcome. The xG features fill it with "wrist".
- **Coordinates:** center ice is (0, 0), goal lines at x = ±89, blue lines at x = ±25. Teams switch ends each period; normalize per game, period and team.

Rebound and Rush Definitions (`shot_context.py`)

NST conventions, with the no-stoppage rule enforced (faceoffs, whistles, penalties and goals break the chain):

- **Rebound:** unblocked shot within 3 s of a prior attempt by the same team. 9.4% of shots; converts at 12.4% vs 6.6% for other shots.
- **Rush:** unblocked shot within 4 s of any event in the neutral zone or the shooter's defensive zone. 2.7% of shots; weak as a yes/no flag (7.4% vs 7.1%), but strong when split by the previous event: takeaway 10.1%, blocked shot 11.1%, giveaway 9.0%, hit 4.0%.
- NST spot-check: 1 of 3 rush flags matched in game 2025020993. Both misses had an opponent's shot as the precursor, at physically impossible 1–2 s gaps (about 17% of rush flags, 0.5% of training data).

Parallel Build: Track A vs Track B

The honest version of the model is the PyMC hierarchical NUTS sampler. But porting your AEM 8290 AM MCMC structure to a tiny hockey problem first buys real value: a reference implementation you trust, and an independent cross-check on the PyMC model.

Track A — Toy AM MCMC (Soil Pipeline Port)

Scope: one team pair at a time. Parameter vector:

$$\theta = \left(\mu_{\text{off,A}}, \mu_{\text{def,A}}, \mu_{\text{off,B}}, \mu_{\text{def,B}} \right)$$

Likelihood: Poisson on observed goal counts in head-to-head games:

$$p\left(y^{\text{obs}} \mid \theta \right) = \prod_{g} \text{Poisson}\left(G_{\text{home},\, g} \mid \lambda_{\text{home}}\left(\theta \right) \right) \cdot \text{Poisson}\left(G_{\text{away},\, g} \mid \lambda_{\text{away}}\left(\theta \right) \right)$$

Priors: bounded uniform in standardized $\xi \in \left[-1, 1 \right]^{4}$ space, same as soil.

Identifiability — only two of the four parameters are learnable. With $\lambda_A = \exp\left(\alpha_0 + \mu_{\text{off},\, A} - \mu_{\text{def},\, B} \right)$ and $\lambda_B = \exp\left(\alpha_0 + \mu_{\text{off},\, B} - \mu_{\text{def},\, A} \right)$, adding the same constant to $\mu_{\text{off},\, A}$ and $\mu_{\text{def},\, B}$ (or to $\mu_{\text{off},\, B}$ and $\mu_{\text{def},\, A}$) leaves the likelihood unchanged. The data pin down only the two differences

$$d_A = \mu_{\text{off},\, A} - \mu_{\text{def},\, B}, \qquad d_B = \mu_{\text{off},\, B} - \mu_{\text{def},\, A}$$

and the rest of the posterior is the bounded prior. Either sample (d_A, d_B) directly, or keep four parameters and report and compare only d_A, d_B. Fix α_0 to the log of the league-mean goals per team-game from the training data (about $\log 3$), so d_A, d_B are on the same scale as Track B.

Expect wide posteriors. Two teams meet three or four times a season, so even five seasons give only about 15 games per pair. Track A is a correctness check, not a source of useful team ratings.

Sampler: Adaptive Metropolis, three phases:

1. **Burn-in** ($n_b = 5000$): isotropic proposal $\Sigma_p = \sigma_0^2 I$ with $\sigma_0 = 0.1$
2. **Adapt:** estimate posterior covariance from burn-in, set $\Sigma_p = s \cdot \widehat{\Sigma}$ with $s = (2.38)^2/d$
3. **Sample:** main run with fixed adapted covariance

Reuses: your soil sampler module, standardization machinery, diagnostics (trace plots, ACF, acceptance rate, R-hat).

Purpose: proof of life, reference posteriors, bug-catching for Track B.

Track B — Hierarchical PyMC Model

Scope: full league simultaneously.

Model:

$$
\begin{aligned}
\mu_{\text{off},\,t} &\sim \mathcal{N}\left(0, \sigma_{\text{off}}^2\right) \quad \text{for each team } t \\
\mu_{\text{def},\,t} &\sim \mathcal{N}\left(0, \sigma_{\text{def}}^2\right) \quad \text{for each team } t \\
\gamma_{\text{goalie},\,g} &\sim \mathcal{N}\left(0, \sigma_{\text{goalie}}^2\right) \quad \text{for each goalie } g \\
\alpha_0 &\sim \mathcal{N}\left(\log 3,\ 0.5^2\right) \quad \text{league baseline scoring rate} \\
\alpha_{\text{home}} &\sim \mathcal{N}(0, 1) \\
\log \lambda_{\text{home}} &= \alpha_0 + \alpha_{\text{home}} + \mu_{\text{off},\,h} - \mu_{\text{def},\,a} - \gamma_{\text{goalie},\,a} \\
\log \lambda_{\text{away}} &= \alpha_0 + \mu_{\text{off},\,a} - \mu_{\text{def},\,h} - \gamma_{\text{goalie},\,h} \\
G_{\text{home}} &\sim \text{Poisson}\left(\lambda_{\text{home}}\right) \\
G_{\text{away}} &\sim \text{Poisson}\left(\lambda_{\text{away}}\right)
\end{aligned}
$$

Hyperpriors on $\sigma_{\text{off}}, \sigma_{\text{def}}, \sigma_{\text{goalie}}$ — typically half-normal or half-Cauchy.

Why the global intercept α_0 is required. Every team and goalie effect has a zero-mean prior, so without α_0 the model is centered at $\log \lambda = 0$, i.e. about 1 goal per team per game instead of about 3. The team effects would have to shift together to make up the difference, fighting their own priors. α_0 carries the league scoring level; the team and goalie effects are then deviations from it. Its prior is centered on $\log 3 \approx 1.1$ and is wide enough to let the data decide.

Remaining soft identifiability. α_0 can still trade off against the average of the μ_{off}'s, the μ_{def}'s and the γ's. The zero-mean priors resolve this softly. If α_0 mixes poorly or correlates strongly with the team means, add sum-to-zero constraints on μ_{off} and μ_{def} (e.g. `pm.ZeroSumNormal`).

Sampler: NUTS. Write every random effect in non-centered form from the start ($\mu_{\text{off},\,t} = \sigma_{\text{off}}\, z_t$, $z_t \sim \mathcal{N}(0, 1)$), because centered hierarchical models with small σ produce funnel geometry and

divergences.

Output: ArviZ `InferenceData` with full joint posterior across all teams and goalies.

Purpose: the actual production model.

Cross-Validation Between Tracks

The two tracks only *should* agree when they are fit to the same data with the same likelihood. Comparing Track A on head-to-head games with full Track B on the whole league compares different datasets and different priors, so a gap between them proves nothing. The check is therefore done in three steps, from strict to loose.

1. Synthetic recovery (both tracks). Simulate a full league season from Track B's generative model with known parameters.

- Fit Track B and check that the posteriors cover the true team and goalie effects.
- Fit Track A on 2–3 team pairs from the same simulated games and check that its d_A, d_B intervals cover the true differences.

2. Like-for-like on real data (the strict agreement test). For 2–3 team pairs, fit Track B in a *matched* configuration on the same (A, B) head-to-head games Track A sees:

- no pooling ($\sigma_{\text{off}}, \sigma_{\text{def}}$ fixed large),
- goalie and home terms switched off,
- α_0 fixed to the same value as Track A.

The two posteriors on d_A, d_B should then match to within Monte Carlo error. A real disagreement here means a bug in the likelihood, the parameterization or one of the samplers.

3. Pooled Track B vs Track A (qualitative only). Fit full Track B on league data and compute its posterior on the same d_A, d_B. It should be much narrower and shrunk toward zero, and it should usually sit inside Track A's wide interval. A pooled interval far outside Track A's is a flag to investigate, not an automatic failure, since pooling and the extra league data are supposed to move it.

Build Sequence

Realistic for a side project with focused sessions a few times a week. Roughly two months to a usable single-game predictor.

Phase 1 — Data Foundation (Weeks 1–2) — DONE

- ~~Expand `scripts/load_shots.py` to iterate over the full league schedule.~~ Done as `scripts/load_all.py`: all 32 teams, 5 seasons.
- ~~Verify shot data integrity.~~ `notebooks/data_sanity.ipynb`, sections 1–13, all passing.
- ~~Backfill 2–3 seasons.~~ 2021-22 through 2025-26 (6,560 games).

Added beyond the original plan:

- `game_events` table (full play-by-play, every event type), needed for rush detection.
- `shot_context` table with rebound / rush flags, prior event type and time since prior event.
- Fixes found by the checks: goal-shooter repair, blocked-shot zone frame, no-stoppage rule (see Data Contract → Known Data Quirks).
- External check against Natural Stat Trick (`scripts/nst_spotcheck.py`, `scripts/rush_event_audit.py`).

Phase 2 — xG Model (Week 3) — DONE

Files:

- `src/processing/xg_processing.py` — `build_xg_features(conn)`: one row per shot, 13 features
- `src/models/deterministic/xg.py` — `ExpectedGoalsModel` with `.fit()`, `.predict()`, `.evaluate()`, `.calibration()`, `.feature_importance()`, `.save()`, `.load()`
- `src/pipelines/train_xg.py` — run with `py -m src.pipelines.train_xg [--final]`
- `notebooks/xg_features_check.ipynb`, `notebooks/xg_model_dev.ipynb`

Features (13): distance, angle, shot type (grouped), rebound flag, rush flag, previous event type, time since previous event, skater difference, total skaters, own goalie pulled, distance from previous event, speed from previous event, rebound angle change.

Model: LightGBM, log-loss objective. `num_leaves=31`, `min_child_samples=200`, `learning_rate=0.03`, `subsample=0.8`, `colsample_bytree=0.8`. A 9-combination grid search found no better setting (top six within 0.00005 log loss; 63 leaves worse).

Season roles: train 2021-22 to 2023-24, validate 2024-25 (early stopping picks 413 trees), then refit on 2021-22 to 2024-25 and test once on 2025-26. Recency weights 1.0 / 1.25 / 1.5 / 2.0, oldest to newest. Random seed 42.

Results on the 2025-26 test season:

	Baseline (distance + angle, logistic)	**Final LightGBM**
AUC	0.714	**0.769**
Log loss	0.2307	**0.2167**
Predicted vs actual goals	+1.4%	+2.5% (7,770 vs 7,578)
Calibration	Mid-range under, top over	Within 0.8 points in every decile

What moved the score, in order: LightGBM over logistic regression; previous-event features; recency weighting; shot-type grouping (fixed the goal total).

MoneyPuck comparison (2025-26, 5-on-5, team-season totals, 32 teams): xGF r = 0.970, xGA r = 0.940, total xG ratio 1.000. MoneyPuck blocks scripted downloads; save `teams.csv` by hand to `data/external/`.

Saved models (`output/models/`), version v1:

- `xg_lgbm_v1_20212022_20242025.pkl` + `_metrics.json` — the evaluated model
- `xg_lgbm_v1_live.pkl` + `_metrics.json` — same settings, trained on all five seasons, for scoring new games
- `XG_MODEL_PATH` in `config/settings.py` names the active model; load it with `load_active_model()`.
- Metrics files record version, seed, settings, features, training seasons, data cutoff and library versions.
- `train_xg.py` refuses to overwrite an existing file; bump `MODEL_VERSION` in `xg.py` when features or settings change.

Gaps against the original plan:

- MoneyPuck validation was done on team-season totals, not per-game totals. Per-game is stricter and still to do.
- Method is named `.predict()`, not `.predict_proba()`.

Phase 3 — Team Strength & Goalie Aggregates (Week 4) — DONE

Files:

- `src/pipelines/score_xg.py` — scores every shot into the `shot_xg` table. Run with `py -m src.pipelines.score_xg [--season YYYYYYYY]`.
- `src/processing/team_strength.py` — `team_game_xg(conn)`, `rolling_strength(tg, window=25)`, `strength_as_of(tg, date)`
- `src/processing/goalie.py` — `goalie_game_xg(conn)`
- `src/models/stochastic/goalie.py` — `goalie_rating_as_of(gg, date, k=800, lookback_days=730, season_adjust=True)`
- `notebooks/phase3_check.ipynb` — correctness checks plus the tests that chose the window and `k`

Scoring every shot (`shot_xg`). Each season 2021-22 to 2025-26 is scored by a leave-one-season-out model (trained on the other four seasons, 413 trees, same settings as v1), so no shot is scored by a model that saw it. Seasons the live model never trained on (2026-27 onward) are scored by the live model. xG / goals by season: 1.032, 1.031, 0.974, 0.969, 1.024 (see *Season-level xG offset* in Open Questions).

Team strength. One row per team per game (`team_game_xg`): xGF, xGA, goals for and against (periods 1–4, empty-net goals included) and game length in minutes (overtime counted). `rolling_strength` adds each team's estimate going *into* every game from its previous 25 games: xGF/60 and xGA/60 as total xG ÷ total minutes × 60, and their variance (game-to-game variance ÷ games). Windows span seasons. `strength_as_of` gives the same estimate for every team on any date.

Goalie rating. `goalie_game_xg` credits each shot to the goalie in net, so a pulled starter and his replacement each carry only the shots they faced. The rating is Gamma-Poisson on goals allowed per expected goal:

$$GA \sim \text{Poisson}\,(\,\theta \cdot xGA\,)\,, \qquad \theta \sim \text{Gamma}\,(\,k, k\,)\,, \qquad \theta \mid \text{data} \sim \text{Gamma}\,(\,k + GA,\; k + xGA\,)$$

$\theta < 1$ is better than average. Each season's xGA is first rescaled so league GA ÷ xGA = 1.0 within that season (using only games before the rating date). Output per goalie: posterior mean and variance of θ, a 90% credible interval, raw GSAx and games. This is the swappable goalie interface; on Track B's additive log scale, $\gamma_g \approx -\log \theta_g$ (positive = better).

Results (`phase3_check.ipynb`):

Correctness: two rows per game (13,120), league xGF = league xGA, xGF and goalie xGA per season match `shot_xg` exactly, every goalie-game named, and a hand-computed pre-game estimate matches `rolling_strength` (no leakage).

Team strength predicting the next game (6,381 games where both teams had 10+ games of history):

Window	Correlation with goal diff (xG)	(goals)	Picks winner (xG)
10	0.233	0.168	58.2%
25	**0.251**	0.206	**59.3%**
50	**0.251**	0.240	59.1%

xG-based estimates beat goals-based ones at every window. 25 and 50 tie; 25 is kept because it reacts faster.

Goalie rating predicting the next season (2022-23 to 2025-26 pooled, 253 goalie-seasons with 20+ games, correlation margin of error about ±0.12):

k	Weighted error (adjusted)	Correlation (adjusted)	Correlation (unadjusted)
1 (raw record)	0.0284	0.037	0.043
100	0.0138	0.100	0.094
400	0.0120	0.120	0.105
800	**0.0119**	0.124	0.106
1600	0.0119	0.125	0.106

The season adjustment improves error and correlation from k = 100 up. Error is lowest at k = 800 and flat from about 400 to 1,600. The signal is real but weak: a goalie's past two seasons are a modest guide to his next one, consistent with public hockey analytics. With k = 800, ratings for big-sample goalies span roughly 0.96 to 1.04.

Settings kept: team window 25 games; goalie `k = 800`, `season_adjust=True`, `lookback_days = 730`.

Notes:

- `games` in goalie output counts appearances, so a short relief stint counts as one game. The rating itself is weighted by xGA faced, not games.
- Utah (`UTA`) has a different abbreviation from Arizona, so its team-strength history starts fresh in 2024-25.
- Goalie names come from `goalie_game_stats` (boxscores), which covers every goalie who played, not only current rosters.

Phase 4 — Track A Toy MCMC (Week 5)

- Port your AM sampler from soil to `src/models/stochastic/am_sampler.py`.

- Write `src/models/stochastic/toy_mcmc.py`: 4-parameter Poisson model for one team pair, with α_0 fixed and results reported on the identified differences d_A, d_B.
- Write `src/pipelines/fit_toy.py`: load team pair data, run AM, save posterior samples (ArviZ NetCDF for consistency with Track B), produce diagnostics (trace plots, ACF, acceptance rate).
- Validate: synthetic recovery on simulated data with known θ^*, like the soil project's posterior recovery check.

Phase 5 — Track B Hierarchical PyMC (Week 6)

- Write `src/models/stochastic/hierarchical.py` with the full league model: global intercept α_0, non-centered random effects, and a switch for the matched no-pooling configuration used in the cross-check.
- Write `src/pipelines/fit_hierarchical.py`: load full-season game data, fit, save InferenceData.
- Diagnostics: R-hat, ESS, divergences, energy plots.
- Cross-validate against Track A on 2–3 team pairs using the three steps in *Cross-Validation Between Tracks* (synthetic recovery, matched like-for-like fit, then pooled comparison).

Phase 6 — Game Model + Monte Carlo (Week 7)

- Write `src/models/game_model.py`: function taking $\left(\mu_{\text{off}}, \mu_{\text{def}}, \gamma_{\text{goalie}}, \text{home flag}\right)$ samples and producing $\left(\lambda_{\text{home}}, \lambda_{\text{away}}\right)$ samples.
- Write `src/models/stochastic/montecarlo.py`: draws posterior samples, runs game model, samples Poisson goal counts, handles OT/SO.
- Write `src/pipelines/predict_game.py`: end-to-end orchestration. Takes two teams + date, returns predictive distribution.
- Write `src/analysis/aggregation.py`: reduce sim output to $P(\text{ win })$, credible intervals on score margin, etc.

Phase 7 — Validation (Week 8)

- `src/validation/scoring.py`: log-loss, Brier score, CRPS for goal totals.
- `src/validation/calibration.py`: reliability diagrams with bootstrap confidence bands.
- `src/validation/backtest.py`: walk-forward harness. For each held-out game in chronological order, fit components only on prior data, predict, score.
- `src/pipelines/backtest_season.py`: run the full validation on a held-out season tail.

Phase 8 — PCE for Sobol (Week 9+)

- `src/models/stochastic/pce.py`: port PCE machinery from soil. Legendre basis (uniform inputs), total-order truncation, non-intrusive regression.
- Fit on team-level input ranges (e.g., xGF/60 ∈ [2.0, 3.5], goalie GSAx ∈ [-0.3, 0.5]) with $P(\text{ home win })$ as the QoI.
- `src/analysis/sensitivity.py`: analytical Sobol indices from PCE coefficients (the "sensitivity for free" trick).
- Cross-check with Saltelli sampling on the surrogate.

v2 — Planned Improvements

Everything in this section comes **after Phase 7**: the v1 team model has to be built and backtested first, because it is the baseline each improvement must beat. Each item is kept only if it improves held-out log loss in the Phase 7 walk-forward backtest.

Item	What it adds	Main payoff
v2.1 Hierarchical xG layer	Shooter, goalie, rink and season × shot-type effects on top of LightGBM xG	Finishing talent, a shrunk xG-based goalie rating, fixes for arena bias and redirect drift
v2.2 Lineup model	Per-skater offensive/defensive effects (Bayesian RAPM), aggregated over tonight's lineup	Predictions that react to injuries, scratches and trades
v2.3 Other candidates	Score/rest/travel effects, better OT/SO, goal dependence, etc.	Smaller calibration gains

v2.1 — Hierarchical xG Layer

Why a layer, not a fully Bayesian xG model. LightGBM already pins down the shot-geometry function: with 562k shots, uncertainty in the distance/angle/shot-type curve is negligible next to game randomness and team-strength uncertainty. A fully Bayesian xG model would be slow on 562k shots and would need its nonlinear interactions written by hand, likely losing AUC. What LightGBM *can't* do is tell shooters, goalies and rinks apart, or let shot-type conversion drift by season. A small hierarchical model on top of it adds exactly that.

Model. LightGBM's prediction enters as a fixed offset; only the random effects are estimated:

$$
\begin{aligned}
\operatorname{logit} p_i &= \operatorname{logit}\left(xG_i^{\text{lgbm}}\right) + \beta_{s[i]} - \gamma_{g[i]} + \delta_{r[i]} + \varepsilon_{k[i]} \\
\beta_s &\sim \mathcal{N}\left(0, \sigma_\beta^2\right) && \text{shooter finishing talent} \\
\gamma_g &\sim \mathcal{N}\left(0, \sigma_\gamma^2\right) && \text{goalie skill (positive = better, same sign as Track B)} \\
\delta_r &\sim \mathcal{N}\left(0, \sigma_\delta^2\right) && \text{rink / arena recording effect} \\
\varepsilon_k &\sim \mathcal{N}\left(0, \sigma_\varepsilon^2\right) && \text{season} \times \text{shot-type group} \\
y_i &\sim \text{Bernoulli}\left(p_i\right)
\end{aligned}
$$

Half-normal hyperpriors on the four σ's. Partial pooling shrinks low-volume shooters and goalies toward zero; high-volume ones keep their own estimate.

What each term addresses:

- β — finishing talent. Feeds player point distributions later.
- γ — a shrunk, xG-adjusted goalie rating. Replaces the Phase 3 conjugate rating if it backtests better.
- δ — the arena recording bias from Open Questions. A logit intercept per rink is an approximation: the real bias is in recorded *location*, so a distance-adjustment version is a later

refinement. Home teams play 41 games in their own rink, so δ is partly confounded with home shooters and goalies; fitting all terms jointly is what separates them.
- ε — the redirect location drift. Lets conversion by shot type move season to season instead of relying only on retraining.

Inputs — the one hard rule. The offset must come from the walk-forward `shot_xg` table (Phase 3), never from in-sample LightGBM predictions. In-sample xG is overfit, its residuals look too small, and the random effects would be shrunk toward zero.

Fitting.

- NumPyro NUTS (or PyMC with the NumPyro backend). Plain PyMC on 562k Bernoulli rows is slow.
- Non-centered parameterization for every random effect from the start.
- Parameter count is a few thousand (shooters dominate) — fast once the offset does the heavy lifting.

Files:

- `src/models/stochastic/xg_hierarchical.py` — model definition.
- `src/pipelines/fit_xg_layer.py` — load `shot_xg` + IDs, fit, save `output/posteriors/xg_layer_<season>.nc`.
- Adjusted per-shot xG (posterior mean and sd) written to a `shot_xg_v2` table; `shot_xg` stays untouched.

Validation — keep the layer only if it earns it:

1. **Synthetic recovery.** Simulate shots on top of real offsets with known $\beta, \gamma, \delta, \varepsilon$; check recovery, same discipline as the soil project.
2. **Held-out shot log loss.** Fit on seasons up to $t-1$, score season t; must beat the LightGBM offset alone.
3. **Posterior predictive checks.** Goals per shooter, per goalie and per rink against their predictive intervals.
4. **Sanity of the spreads.** σ_β and σ_γ should be small (finishing and goaltending talent are real but modest on the logit scale).
5. **Game-level payoff.** Swap γ into the game model in place of the Phase 3 goalie rating and rerun the Phase 7 backtest. Keep whichever gives better game log loss.

Open design question — goalie double-counting. Track B also has a γ_{goalie} term. If the game model uses the xG-layer goalie effect, either drop Track B's goalie term or use the xG-layer posterior as its prior. Don't let goalie skill enter twice.

v2.2 — Lineup (Player-Level) Model

Idea. The v1 game model rates *teams*. It can't tell when a top player is injured, scratched or traded. The lineup model rates each *skater's* effect on xG for and against, then builds tonight's team strength from tonight's dressed lineup. It is essentially Bayesian RAPM, fit with the same hierarchical/NUTS machinery as Track B.

Data unit: the stint. A stint is a stretch of time during which the set of players on the ice doesn't change; any line change by either team ends it. `stints.py` builds them by overlaying the 12 on-ice players' intervals from `shifts` and cutting at every change. Each stint records duration, the 10 skaters and 2 goalies, strength, score state, zone start, and shots and summed xG for each side. Expect a few hundred stints per game, on the order of a couple million over five seasons.

Model (sketch). For stint j with duration t_j:

$$\text{rate}_{\text{home},\,j} = \alpha_0 + \alpha_{\text{home}} + \sum_{p \,\in\, \text{home skaters}} o_p - \sum_{q \,\in\, \text{away skaters}} d_q + (\text{score, zone terms})$$

and the mirror image for the away side. Each skater gets an offensive effect o_p (raises his team's xG rate) and a defensive effect d_p (suppresses the opponent's). Partial pooling learns the prior widths instead of tuning a ridge penalty, and the joint posterior shows when linemates can't be told apart (their effects come out negatively correlated).

From ratings to a game prediction.

1. Take tonight's dressed lineup (known about an hour before puck drop).
2. Project each player's ice time from **past games only**.
3. Team xGF/60 and xGA/60 ≈ ice-time-weighted sums of the dressed players' o_p and d_p.
4. Convert xG to goals with finishing and goaltending effects (tonight's starting goalie; later the v2.1 shooter β and goalie γ).
5. Feed $\lambda_{\text{home}}, \lambda_{\text{away}}$ into the same Phase 6 game model and Monte Carlo. Nothing downstream changes.

Choices and tentative defaults. None of these need deciding until Phase 7 is done; the defaults are starting points, not decisions.

Before starting

#	Choice	Options	Tentative default
1	Order within v2	Lineup model first, xG layer first, or both	xG layer first — smaller, and its β, γ feed step 4
2	Ridge RAPM reference	Yes / no	Yes — fit `deterministic/rapm.py` with scikit-learn first and check the Bayesian means against it

Model design

#	Choice	Options	Tentative default
3	Likelihood	Weighted Gaussian on xG/60; Tweedie on summed xG; two-part (shot counts + shot quality)	Weighted Gaussian (variance $\propto$ 1/duration); upgrade later
4	Scale of player effects	Linear on the xG rate, or log scale (effects multiply)	Linear — comparable to public RAPM
5	Game situations	5v5 only, or separate PP/PK player effects	5v5 only; special teams stay team-level
6	Stint covariates	Score state, zone start, home ice, back-to-backs	Score state, zone start, home ice
7	Prior structure	One shared σ; separate forward/defense σ; prior centered on last season	Separate F/D σ; no carryover in the first version
8	Data window	Rolling 2–3 seasons, or all 5 with recency weights	Rolling 2 seasons
9	Goalies in the stint model	In or out	Out — goalies affect whether shots go in, not shot quality; they enter at step 4
10	Fitting method	NumPyro NUTS, or variational / Laplace approximation	NumPyro NUTS, sparse design matrix; approximate only if too slow

Making a prediction

#	Choice	Options	Tentative default
11	Ice-time projection	Last-N-games average share, or line-based deployment	Last 10 games, rescaled so the dressed lineup sums to a full game
12	Lineup source in the backtest	Actual dressed lineup vs. projected lineup	Actual dressed lineup (known pregame) with **projected** TOI — never that game's real TOI
13	xG → goals	League-average finishing + Phase 3 goalie rating, or v2.1 β, γ	League average + Phase 3 goalie first; swap in v2.1 later

Evaluation

#	Choice	Options	Tentative default
14	What counts as winning	Better overall game log loss, or better on games with major lineup changes	Score both. Fix the "major change" threshold in advance (e.g. a top-6 F or top-4 D out)

Validation. Same Phase 7 walk-forward harness as the team model: game log loss and Brier, overall and on the lineup-change subset. Plus synthetic recovery on simulated stints with known player effects. If the lineup model only wins on lineup-change games, consider a cheaper hybrid: the team model adjusted for known missing players.

Files. `src/processing/stints.py`, `src/models/stochastic/lineup_rapm.py`, `src/pipelines/fit_lineup.py`, posteriors to `output/posteriors/lineup_<window>.nc`.

v2.3 — Other Candidate Improvements

Smaller items already listed under *Open Questions*, gathered here so v2 has one list:

- **Score effects, rest, travel, back-to-backs** as game-model covariates.
- **OT/SO model** weighted by goalie quality instead of a flat 50/50.
- **Goal dependence:** Dixon-Coles correction or a bivariate model, instead of independent home/away Poisson.
- **Arena bias, location version:** adjust recorded shot distance per rink, refining the v2.1 rink intercept.
- **Rebound and rush windows:** test 2/3/4 s by retraining xG and comparing validation log loss.
- **Player-level point distributions:** built on the v2.2 player effects plus the v2.1 shooter finishing effects.

Key Conceptual Notes

Cheap vs Honest Uncertainty Propagation

Both versions produce joint vector samples. The difference is *where the correlations live.*

Independent marginal sampling (cheap, fragmented). Each component fit independently, sampled independently. Joint distribution is the product of marginals → diagonal covariance. Misses correlations between parameters that exist in the data (e.g., team offense and defense are correlated; team strength and goalie skill are negatively correlated for bad-team-good-goalie scenarios).

Joint hierarchical posterior (honest). All parameters fit jointly in a single hierarchical model. Sample the whole vector at once → full covariance, possibly non-Gaussian. This is what Track B produces.

Cheap version typically miscalibrates the tails. Won't catastrophically break win probabilities but will produce wrong probabilities for extreme outcomes.

Why PCE Earns Its Keep for Hockey

For the soil project, PCE was essential because the truth model was ~1 s/eval. For the hockey team-level game model, evaluation is in microseconds — direct MCMC on the truth model is viable, surrogate not strictly needed for inference.

PCE earns its keep in hockey for **sensitivity analysis**: analytical Sobol indices directly from PCE coefficients via the variance-decomposition identity

$$\mathrm{Var}\left[\widehat{Y}\right] = \sum_{|\alpha|>0} c_\alpha^2 \mathbb{E}\left[\Psi_\alpha^2\right]$$

This gives a clean decomposition of outcome variance into contributions from each input (team xGF/60, team xGA/60, goalie GSAx, etc.) — without running Saltelli on millions of samples.

If you later upgrade to a possession-by-possession simulator that's slow to run, PCE returns as an inference-speed tool.

Sources of Uncertainty (Ranked by Sobol Later)

Three categories injected at different layers:

1. **Aleatoric (irreducible).** Game randomness — Poisson sampling in the game model. Even with perfect knowledge of λ, the puck still bounces.
2. **Parameter uncertainty.** Posterior width on team/goalie/player parameters from finite-sample inference.
3. **Model misspecification.** The Poisson form, the independence assumption between home/away goals, the missing factors (rest, travel, injuries). Hardest to quantify; partially addressed by Dixon-Coles corrections and held-out validation.

Sobol analysis at the end ranks which of these matter most for which outcomes.

Persistence & Reproducibility

- **xG model:** pickle under `output/models/`, named with `MODEL_VERSION` (`xg_lgbm_v1_*`), plus a `_metrics.json` with version, seed, settings, data cutoff and library versions. `XG_MODEL_PATH` in `config/settings.py` points to the active model.
- **Per-shot xG:** the `shot_xg` table, rebuilt by `score_xg.py`; each row records the model version and which model scored it.
- **Posterior samples:** ArviZ NetCDF under `output/posteriors/`. Both tracks write to the same format so downstream code is sampler-agnostic.
- **PCE surrogate:** pickle with input-range metadata and coefficient array.
- **Backtest results:** CSV under `output/backtests/` with one row per held-out game (date, teams, prediction, outcome, log-loss contribution).

Reproducibility checklist for any fit:

- Random seed logged
- Data cutoff date logged
- Model version logged
- Software versions (PyMC, NumPy, LightGBM) logged in metadata

---

Open Questions Deferred to "When We Get There"

These are real decisions but premature to make now:

- **OOP vs functional structure** for the inference layer. Decide once 2–3 modules are written and friction becomes obvious.
- **Whether to use `__init__.py` re-exports** for cleaner imports. Default to empty files; refactor later if imports get ugly.

- **OT/SO model.** Start with a flat 50/50 conditional on tied regulation. Upgrade to a goalie-quality-weighted shootout model later if it materially changes calibration.
- **Score effects, rest, travel, back-to-backs.** Real factors, but adding them prematurely complicates the base model. Document as v2 features.
- **Player-level point distributions.** Requires shift-level data and per-player rate modeling. Defer until team-level pipeline is calibrated; planned on top of the v2.2 lineup model.

Data and model items found in Phases 1–3:

- **Redirect location drift.** Each season, redirects are recorded closer to the net (share under 10 ft: 24.8% in 2021-22 → 38.2% in 2025-26) while their goal rate falls (9.4% → 7.5%). The model overrates 2025-26 redirects by about 24%, and this accounts for most of the +2.4% total bias. Recency weighting helps only slightly. Retrain every season. The v2.1 season × shot-type effect (ε) is the planned fix.

- **Arena recording bias.** Some rinks log shot locations closer or farther than others. MoneyPuck adjusts for it; this model does not. Planned fix: rink effect (δ) in v2.1.

- **Timestamp artifacts.** The feed logs whole seconds, so an opponent's shot can appear 1–2 s before a shot at the other net. Affects about 0.5% of shots.

- **NST rush definition.** NST appears not to count an opponent's shot as a rush precursor. Unconfirmed at three events; a query-time filter gives NST parity if needed.

- **Rebound and rush windows.** 3 s and 4 s are NST conventions, not fitted. Test 2/3/4 s by retraining and comparing validation log loss.

- **Previous-event location for stoppage-type events** is missing for about 0.5% of shots.

- **Season-level xG offset.** Leave-one-season-out xG runs about 3% high in 2021-22 and 2022-23 and about 3% low in 2023-24 and 2024-25 (xG/goals 1.032, 1.031, 0.974, 0.969, 1.024). Normal luck would move this about 1%, so goal conversion is changing between seasons more than shot quality explains (goaltending trends, recording changes, play the features miss). Within-season team comparisons are unaffected. Goalie ratings now handle it (`season_adjust=True` rescales each season to its own league average). The Phase 6 game model still needs a per-season league-average term or the same rescaling for team xG.

- **Early-season team strength.** v1 rolling windows span seasons (the last 25 games, whatever season), so every team has an estimate on opening night but off-season roster changes are ignored. Refinement: pull early-season estimates toward the league average.

- **When to retrain the xG model.** Keep the live model fixed during a season, so it never scores games it trained on. After the season ends, add it, move `CURRENT_SEASON` / `LAST_SEASON` forward one year, and run `train_xg.py --final` and `score_xg.py`.

- **Rename `CURRENT_SEASON` to `TEST_SEASON`.** It is used as the xG test season, not the season being played, which is easy to confuse.

- **Goalie signal is weak.** Adjusted next-season correlation is 0.125 (±0.12), so goalies should move game predictions only modestly. Repeat the `k` test when a new season is added; the best

value sat on a flat stretch (400–1,600).

- **Team strength ignores schedule strength.** The rolling window doesn't adjust for the quality of opponents faced. Track B's joint fit handles this; the v1 rolling estimate does not.
- **Per-game MoneyPuck comparison** is still to do (Phase 2 gap).

Decisions Made

For future reference, the decisions locked in during planning:

- **First shipping target:** single-game win probability.
- **xG approach:** train own model, validate against MoneyPuck.
- **Uncertainty propagation:** honest version (joint hierarchical Bayesian via PyMC NUTS).
- **Build strategy:** Track A (AM MCMC) and Track B (PyMC) in parallel. Track A as integration test and reference; Track B as production.
- **Uncertainty sources to track:** all three (aleatoric, parameter, model misspec); rank with Sobol later, no pre-ranking.
- **Folder structure:** add `analysis/`, `validation/`, `pipelines/` as new top-level under `src/`. `game_model.py` at top level of `models/`.

Decided during Phases 1–2:

- **Raw tables stay raw.** Derived flags live in `shot_context`; filters are applied at query time.
- **xG training set:** unblocked shots, `period <= 4`, empty nets dropped.
- **Season roles:** test = `CURRENT_SEASON`, validation = `LAST_SEASON` (both in `config/settings.py`); the test season is graded once per change, never used to choose settings.
- **Rush in the model:** feed `last_event_type` and `time_since_last_event`; keep `is_rush` for reporting.
- **Strength state:** decoded shooter-relative (skater difference, total skaters, own goalie pulled), not the raw code.
- **Shot types grouped** into `wrist-snap` and `redirect` because the labels changed between seasons.
- **Recency weighting on** (1.0 / 1.25 / 1.5 / 2.0).
- **No recalibration step:** the remaining bias is concentrated in redirects, so a uniform shift would worsen other shot types.
- **Trained `.pkl` files stay out of git;** rebuild with `train_xg.py`.

Decided during Phase 3:

- **Per-shot xG comes from leave-one-season-out models** for historical seasons and from the live model for new ones; stored in `shot_xg`.
- **Team strength:** rolling 25-game window spanning seasons, total xG ÷ total minutes, variance = game-to-game variance ÷ games. Estimates use only earlier games.
- **Goalie rating:** Gamma-Poisson on GA ÷ xGA, `k = 800`, season-adjusted, two-season lookback. Chosen by the next-season prediction test in `phase3_check.ipynb`.
- **Goalie GA excludes empty-net and shootout goals;** team GA/GF include empty-net goals.

- **Build Phase 3 on complete seasons;** load 2026-27 when live predictions are needed. Keep the live xG model fixed during a season.

Decided 2026-10-17:

- **LightGBM stays as the xG base.** Shooter, goalie, rink and season effects go in a v2 hierarchical layer that uses walk-forward LightGBM xG as a fixed offset, not in a fully Bayesian replacement.
- **Track B gets a global intercept** $\alpha_0 \sim \mathcal{N}\left(\log 3, 0.5^2\right)$ and non-centered random effects from the start.
- **Track A reports only identified quantities** (d_A, d_B), with α_0 fixed to the league mean.
- **Track A vs Track B agreement is tested like-for-like** (matched no-pooling Track B on the same games); the pooled comparison is qualitative only.
- **Lineup (player-level) model is v2,** after Phase 7. Its choices are listed in v2.2 with tentative defaults, not decided.
- **One goalie rating.** The Phase 3 goalie rating is built as a swappable interface; the v2 goalie effect replaces it only if it wins the Phase 7 backtest.

Glossary

xG — Expected Goals. Per-shot probability of becoming a goal, summed over shots. Output of a classifier (usually gradient boosting) trained on shot features.

RAPM — Regularized Adjusted Plus-Minus. Ridge regression isolating player impact on shot/goal rates while controlling for teammates and opponents.

GSAx — Goals Saved Above Expected. $xGA - GA$ for goalies. Positive = better than average.

PDO — On-ice $SH\% + SV\%$. Mean-reverts to 100. Luck/regression indicator.

Corsi / Fenwick — Shot attempt differentials. Corsi includes blocks; Fenwick excludes them.

Game Score — Dom Luszczyszyn's weighted-stat composite (~1.0 = average game).

PCE — Polynomial Chaos Expansion. Spectral surrogate for expensive forward models. Orthogonal polynomial basis under input distribution.

Sobol indices — Variance-based sensitivity measures. S_i = fraction of output variance attributable to input i alone; S_{T_i} = total contribution including interactions.

NUTS — No-U-Turn Sampler. HMC variant. Default sampler in modern PyMC.

AM — Adaptive Metropolis. Random-walk MH with covariance estimated from burn-in.

ArviZ — Python library for Bayesian model diagnostics and storage. NetCDF-based `InferenceData` is the standard format.

Walk-forward backtest — Validation strategy where, for each held-out game in chronological order, models are refit using only prior data. Avoids look-ahead bias.

AUC — Area under the ROC curve. The chance the model gives a random goal a higher xG than a random non-goal. 0.5 is guessing; public xG models land around 0.75–0.80.

Log loss — How far predicted probabilities are from what happened. Lower is better; a proper scoring rule.

Calibration — Whether predicted probabilities match observed rates: shots given 10% should score about 10% of the time.

Gradient boosting — Many small decision trees built in sequence, each correcting the previous ones' errors (LightGBM, XGBoost). Contrast with a random forest, which averages independently built trees.

Early stopping — Stop adding trees once the validation score stops improving, to avoid overfitting.

Recency weighting — Counting recent seasons more in training so the model follows current recording habits.

Stint — A stretch of game time during which the set of players on the ice does not change. The unit of data for RAPM.

Tweedie distribution — A compound Poisson–Gamma distribution: a point mass at zero plus a continuous positive part. Fits summed xG per stint.

Offset — A term added to a model's linear predictor with its coefficient fixed at 1, not estimated. In the v2 xG layer, logit(LightGBM xG) is the offset, so the Bayesian model only learns adjustments to it.

Partial pooling — In a hierarchical model, group effects (teams, shooters, goalies) share a common prior whose width is learned. Groups with little data are pulled toward the average; groups with lots of data keep their own estimate.

Non-centered parameterization — Writing a random effect as $\beta_s = \sigma_\beta z_s$ with $z_s \sim \mathcal{N}(0,1)$ instead of sampling $\beta_s \sim \mathcal{N}\left(0, \sigma_\beta^2\right)$ directly. Removes the "funnel" geometry that causes NUTS divergences when σ is small.

Leave-one-season-out — Scoring each season with a model trained on all the other seasons, so no prediction comes from a model that saw that data.

Conjugate prior — A prior that, combined with the likelihood, gives a posterior of the same family, so the update is a simple formula. Gamma is conjugate to Poisson: Gamma(k, k) becomes Gamma(k + GA, k + xGA).

Credible interval — A range that contains the true value with a stated probability under the posterior (e.g. 90%). The Bayesian counterpart of a confidence interval.

Shrinkage — Pulling estimates from small samples toward the average. In the goalie rating, `k` sets how strongly.

Proper scoring rules — Loss functions for probabilistic predictions that are minimized in expectation by the true distribution. Log-loss, Brier, CRPS are proper. RMSE on probabilities is not.